Introduction to Lean

Justus Springer

Imperial College London

Polyhedra in Lean - Berlin 2026

This session

- Short intro to the main components of **Lean**.
- Aimed mostly at beginners.
- But can also serve as reminder for more experienced users.
- **I am not a Lean expert!**

What is Lean?

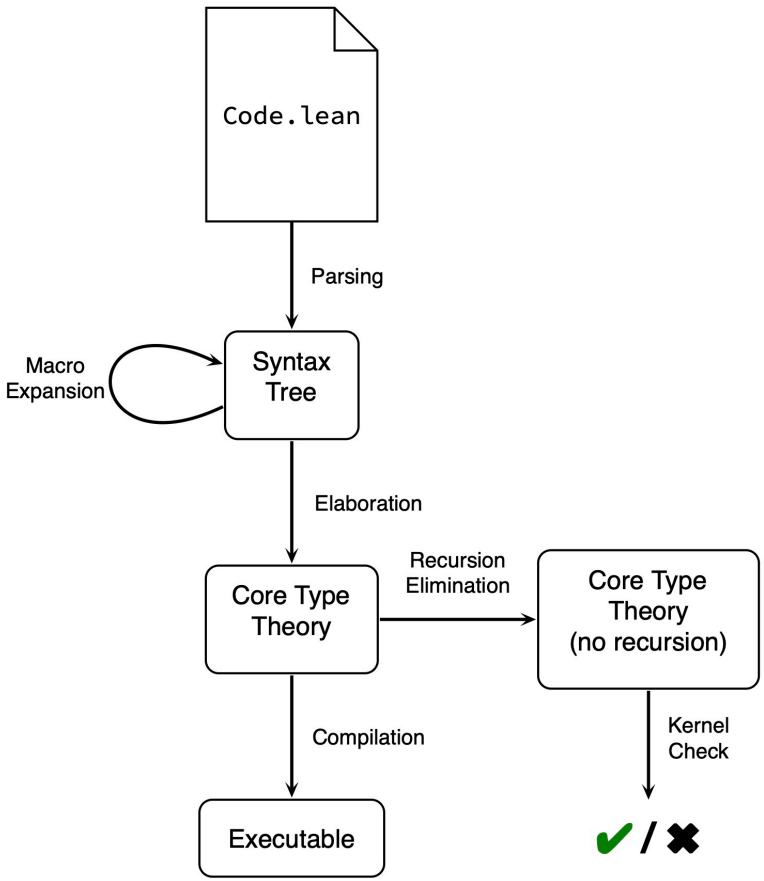
```
def fib : ℕ → ℕ := fun n => match n with
  | 0 => 0
  | 1 => 1
  | n + 2 => fib n + fib (n + 1)

def main : IO Unit := do
  for i in [0:10] do
    IO.println s!"fib {i} = {fib i}"
```

What is Lean?

```
theorem exists_infinite_primes (n : ℕ) : ∃ p, n ≤ p ∧ Prime p :=
  let p := minFac (n ! + 1)
  have f1 : n ! + 1 ≠ 1 := ne_of_gt <| succ_lt_succ <| factorial_pos _
  have pp : Prime p := minFac_prime f1
  have np : n ≤ p :=
    le_of_not_ge fun h =>
      have h₁ : p ∣ n ! := dvd_factorial (minFac_pos _) h
      have h₂ : p ∣ 1 := (Nat.dvd_add_iff_right h₁).2 (minFac_dvd _)
      pp.not_dvd_one h₂
  ⟨p, np, pp⟩
```

The Lean Ecosystem

- **Lean FRO**: Develops and maintains Lean.
  - Current version (Lean 4) released in 2023.
  - Chief Architect: Leo de Moura. Co-founder: Sebastian Ullrich.
- **Zulip**: Main gathering point of the community.
- **Mathlib**: The official mathematics library.
- Many more libraries:



The Lean Pipeline

Macros and Syntax

- **Lean**'s syntax is extremely flexible.
- Users can define **custom syntax** (macros) for their application.

```
∑ x ∈ s, f x           ⇝  Finset.sum s (fun x ↦ f x)
∑ x < n, f x           ⇝  Finset.sum (Finset.Iio n) (fun x ↦ f x)
∑ x ∈ s with p x, f x  ⇝  Finset.sum (s.filter p) (fun x ↦ f x)
∫ x in a..b, f x       ⇝  intervalIntegral (fun x ↦ f x) a b volume
```

- Very powerful, but can result in hard to understand code.

Elaboration

Elaboration is the process of transforming Lean's **user-facing syntax** into its **core type theory**.

During elaboration, Lean will:

- Infer implicit parameters like `{x : α}` from the context,
- Synthesize typeclass arguments like `[Ring R]`,
- Run tactics to create proof terms,
- … and more (it's user-extensible).

Type theory

- Dependent types, i.e. functions whose output type **depends** on the input value:

```
variable (α : Type) (β : α → Type) (f : (x : α) → β x)
```

```
variable (α : Type) (P : α → Prop) (f : ∀ (x : α), P x)
```

- Inductive types, e.g.

```
inductive Nat where
| zero : Nat
| succ (n : Nat) : Nat
```

- Builtin quotient types
- Proof irrelevance: If `P : Prop` and `x y : P`, then `x = y` **definitionally**.

The kernel

- Lean's "trusted" **typechecker**.
- Checks if a term in the core type theory has the claimed type.
- Official one is roughly **7000 lines** of C++.
- Other implementations exist (e.g. in Rust and Lean).
- The kernel is the **only** part of the system you need to trust.
- Every proof has to pass through the kernel.

Two kinds of Equality

- A **meta-property** in Lean's type theory.
- Two terms are **defeq**, if they agree up to "computation".
- Cannot be proved/disproved in Lean.
- Is defined **within** the type theory:

```
inductive Eq {α : Sort*} : α → α → Prop where
  | refl (a : α) : Eq a a
```

- Can be proved/disproved in Lean.
- Definitionally equal implies propositional equal (`by rfl`).

Universes

- What type has `Type`?
- `Type : Type` leads to a **Girard's paradox** (similar to Russell's paradox)
- Fix: Cumulative hierarchy of type universes:

```
Sort 0 : Sort 1
Sort 1 : Sort 2
Sort 2 : Sort 3
...
```

- `Prop := Sort 0` and `Type i := Sort (i + 1)` and `Type := Type 0`.
- `universe u` to introduce a **universe parameter**.
- `Type*` or `Sort*` for an arbitrary type universe.